

Colegio Universitario de Cartago

Curso: Programación V

Profesor: Prof. Ing. Gabriel González Solano

Tarea Corta 2

Investigación Gateways

API Gateway en una arquitectura de microservicios

Integrantes:

Sebastián Jiménez Arrieta

Brandon Steve Guido Navarro

Sebastián Obando Ramírez

Julián Solano Obando

Anthony Emmanuel Gamboa Elizondo

Fecha de entrega: 15 de octubre de 2026

Introducción

En una arquitectura de microservicios cada capacidad del negocio vive en un servicio independiente, con su propia API y su propio ciclo de despliegue. Esa independencia trae una consecuencia práctica: los clientes (aplicaciones web, móviles o de terceros) deberían conocer la dirección y el contrato de cada servicio para poder usarlos. Un API Gateway resuelve ese problema al ofrecer una entrada única al sistema.

Este documento presenta la parte teórica de la Tarea Corta 2. Se define qué es un API Gateway y para qué sirve, se explican los problemas que resuelve y por qué un reverse proxy no siempre es suficiente, se describen diez funciones típicas, se analizan ventajas y desventajas, se revisan patrones relacionados y se compara Ocelot con Kong. Finalmente se indica cuándo conviene usar un Gateway y cuándo evitarlo. Las fuentes consultadas se listan al final en formato APA.

1. Definición de API Gateway y su propósito

Un API Gateway es un componente que funciona como único punto de entrada para los clientes de una aplicación basada en microservicios. Recibe cada solicitud y actúa de dos formas: la reenvía al servicio que corresponde o la resuelve consultando a varios servicios a la vez y devolviendo una sola respuesta (Richardson, s. f.). En términos de ubicación, se coloca entre las aplicaciones cliente y los microservicios (De la Torre, 2018).

Su propósito es separar a los clientes de la estructura interna del sistema. Según Richardson (s. f.), esto los aísla de cómo se particiona la aplicación en servicios y de la ubicación de cada instancia, permite ofrecer una API adecuada a cada tipo de cliente, reduce el número de llamadas de red y traslada al Gateway la lógica de invocar varios servicios. Sin un Gateway, las aplicaciones cliente quedan acopladas a los microservicios internos (De la Torre, 2018).

2. Problemas que resuelve y por qué no basta con un reverse proxy

Richardson (s. f.) describe las fuerzas que motivan el patrón. Estas son las que más pesan en un sistema de microservicios:

- **Granularidad distinta.** Los microservicios exponen APIs de grano fino, así que un cliente que necesita una vista completa debe llamar a muchos servicios.
- **Clientes con necesidades distintas.** Una pantalla de escritorio suele mostrar más datos que una móvil.
- **Redes con rendimiento distinto.** Un cliente móvil tolera pocas llamadas, mientras que una aplicación en servidor puede hacer varias sin afectar la experiencia.
- **Ubicaciones cambiantes.** El número de instancias y su host y puerto cambian dinámicamente.
- **Particionamiento que evoluciona.** La división en servicios puede cambiar y no debería notarse desde afuera.
- **Protocolos diversos.** Algunos servicios pueden usar protocolos poco amigables para la web.

Un reverse proxy recibe tráfico y lo reenvía a un servidor interno según reglas como la ruta o el host. Eso cubre la entrada única y el enrutamiento básico, y en sistemas pequeños puede ser suficiente. Se queda corto cuando además se necesita:

- combinar en una sola respuesta datos de varios servicios, lo que reduce la comunicación excesiva entre cliente y servicios (Microsoft Azure, s. f.);
- exponer una API distinta por tipo de cliente, el patrón Backend for Frontend (Richardson, s. f.);
- aplicar en un solo lugar políticas transversales como seguridad, límites de uso, caché y observabilidad, en vez de repetirlas en cada servicio (Gravitee, s. f.);
- trasladar al Gateway funciones compartidas, como los certificados SSL (Microsoft Azure, s. f.).

Por lo tanto, la diferencia es de alcance. El reverse proxy se concentra en reenviar tráfico, y el Gateway agrega lógica propia de las APIs: composición, adaptación por cliente y políticas.

3. Funciones comunes del Gateway

La tabla 1 describe diez funciones habituales y, cuando las fuentes consultadas lo permiten, indica su presencia en Ocelot y en Kong, las dos herramientas comparadas

en este documento. Donde las fuentes no confirman una función en una herramienta, la celda lo dice expresamente.

Tabla 1

Funciones comunes de un API Gateway

Función	Qué hace	En Ocelot y en Kong
1. Routing / reverse proxy	Dirige cada solicitud al servicio correcto según ruta, método u otros atributos de la solicitud (Apache APISIX, s. f.).	Ocelot: routing es una de sus dos funciones primarias (ThreeMammals, s. f.). Kong: el routing forma parte de su núcleo (Kong Inc., s. f.-c).
2. Autenticación y autorización	Verifica quién es el cliente y si puede ejecutar la solicitud, evitando repetir esa lógica en cada servicio (Richardson, s. f.).	Ocelot: Authentication (ThreeMammals, s. f.). Kong: plugins de autenticación (Kong Inc., s. f.-b).
3. Rate limiting / throttling	Limita cuántas solicitudes puede hacer un cliente en un período, para proteger los servicios.	Ocelot: Rate Limiting (ThreeMammals, s. f.). Kong: plugin de rate limiting (Kong Inc., s. f.-b).
4. Caching	Guarda respuestas para no consultar de nuevo el servicio, lo que reduce latencia y carga.	Ocelot: Caching (ThreeMammals, s. f.). Kong: Proxy Caching (Kong Inc., s. f.-a).
5. SSL termination	El Gateway termina la conexión cifrada y los servicios internos no manejan los certificados (Microsoft Azure, s. f.).	Las fuentes consultadas no detallan esta función en ninguna de las dos herramientas; debe consultarse su documentación oficial.
6. Load balancing	Reparte el tráfico entre varias instancias de un mismo servicio.	Ocelot: Load Balancer (ThreeMammals, s. f.). Kong: Upstreams y Load Balancing (Kong Inc., s. f.-a).
7. Transformación de solicitud / respuesta	Modifica rutas, encabezados o contenido; también traduce entre el protocolo público y el interno (Richardson, s. f.).	Ocelot: Transformations (ThreeMammals, s. f.). Kong: plugins de transformación (Kong Inc., s. f.-b).
8. Agregación / Backend for Frontend	Combina datos de varios servicios en una sola respuesta, o define un Gateway por tipo de cliente (Richardson, s. f.).	Ocelot: Aggregation (ThreeMammals, s. f.). Kong: no aparece entre las capacidades documentadas en las fuentes consultadas.
9. Observabilidad (logs, trazas, métricas)	Registra solicitudes y respuestas para diagnosticar fallas, ya que una operación del cliente puede depender de varias llamadas internas.	Ocelot: Logging (ThreeMammals, s. f.). Kong: logs configurables del Gateway (Kong Inc., s. f.-a).
10. Circuit breaking / resiliencia	Corta las llamadas a un servicio que falla para evitar que el error se propague (Richardson, s. f.).	Ocelot: Quality of Service (ThreeMammals, s. f.). Kong: Health Checks and Circuit Breakers (Kong Inc., s. f.-a).

Nota. Elaboración propia a partir de las fuentes citadas.

4. Ventajas y desventajas

Ventajas

- Aísla a los clientes de cómo se divide el sistema y de dónde están las instancias (Richardson, s. f.).
- Permite ofrecer la API más adecuada para cada cliente (Richardson, s. f.).
- Reduce las llamadas de red al resolver en un solo viaje datos de varios servicios (Richardson, s. f.).
- Simplifica al cliente, porque la lógica de invocar varios servicios pasa al Gateway (Richardson, s. f.).
- Centraliza seguridad y otras políticas transversales, de modo que no se reimplementan en cada servicio (Gravitee, s. f.).

Desventajas y riesgos

- **Mayor complejidad.** El Gateway es una pieza más que hay que desarrollar, desplegar y operar (Richardson, s. f.).
- **Más latencia.** Agrega un salto de red, aunque en la mayoría de las aplicaciones ese costo es pequeño (Richardson, s. f.).
- **Punto único de falla (single point of failure).** Si todo el tráfico pasa por el Gateway y este cae, el sistema deja de ser accesible. Se mitiga con varias instancias y balanceo de carga delante del Gateway.
- **Acoplamiento y cuello de botella organizativo.** De la Torre (2018) advierte que normalmente no es buena idea tener un único Gateway que agregue todos los microservicios de la aplicación, y recomienda límites por cliente (BFF) o por equipo. Microsoft (s. f.-b) añade que un Gateway central único solo conviene si permite áreas de configuración independientes para cada equipo.
- **Carga de lógica indebida.** Microsoft (s. f.-a) sugiere que, si la agregación exige lógica de dominio compleja u orquestaciones largas, esa lógica viva en un servicio dedicado detrás del Gateway y no en el Gateway mismo.

5. Patrones relacionados

Backend for Frontend (BFF)

Es una variación del API Gateway que define un Gateway separado para cada tipo de cliente, por ejemplo web, móvil y terceros, y cada uno expone la API que ese cliente necesita (Richardson, s. f.). Evita una API única que obligue a hacer concesiones a todos los clientes.

Strangler Fig

Fowler (2024) usa la imagen de la higuera estranguladora para describir la modernización gradual de un sistema legado: la parte nueva empieza con pequeñas adiciones, separadas del código antiguo, hasta reemplazarlo. AWS (s. f.) lo resume en tres pasos: transformar, coexistir y eliminar. Aplica cuando hay un monolito que se migra a microservicios, y el Gateway sirve como fachada que decide si una solicitud va al sistema antiguo o al nuevo. En esta tarea no existe un monolito previo, por lo que el patrón se menciona solo como referencia.

API Composition

Consiste en armar una respuesta consultando varios servicios y combinando sus datos. Richardson (s. f.) señala que un API Gateway suele implementarlo, y Microsoft (s. f.-a) lo presenta como Gateway Aggregation: agrupar varias solicitudes individuales en una sola para reducir la comunicación excesiva.

Service Discovery

Mención: el Gateway necesita saber qué instancias de cada servicio están disponibles. Richardson (s. f.) indica que debe usar descubrimiento del lado del cliente o del lado del servidor para enrutar a instancias disponibles. Ocelot lista Service Discovery entre sus funciones (ThreeMammals, s. f.).

6. Comparación de herramientas: Ocelot y Kong

La tabla 2 compara las dos herramientas con los criterios solicitados. La fila de curva de aprendizaje es una valoración orientativa y no una medición, y debe ajustarse con la experiencia real del equipo.

Tabla 2

Comparación entre Ocelot y Kong

Criterio	Ocelot	Kong
Lenguaje / stack	Gateway para .NET, construido como una serie de middlewares de ASP.NET Core; puede trabajar con cualquier servicio que hable HTTP (ThreeMammals, s. f.; De la Torre, 2018).	Aplicación Lua que corre sobre Nginx mediante OpenResty; los plugins se pueden escribir en Lua, Go o JavaScript (Kong Inc., s. f.-c).
Tipo	Open-source, con licencia MIT indicada en su repositorio (ThreeMammals, s. f.).	Open-source (edición OSS) con ediciones y servicios comerciales: Enterprise y Konnect (Kong Inc., s. f.-b).
Curva de aprendizaje	Baja a media para quien ya conoce ASP.NET Core: se instala como paquete NuGet y se configura con archivos JSON (De la Torre, 2018; Microsoft, s. f.-b).	Media: exige aprender sus objetos y plugins, y administrarlo por Admin API o configuración declarativa (Kong Inc., s. f.-c).
Funciones clave	Routing, caching, rate limiting, Quality of Service, load balancer, autenticación, agregación, logging, transformaciones y service discovery (ThreeMammals, s. f.).	Plugins de autenticación, rate limiting y transformaciones (Kong Inc., s. f.-b); load balancing, health checks y circuit breakers, y proxy caching (Kong Inc., s. f.-a).
Caso ideal de uso	Equipos .NET que necesitan un punto de entrada unificado para sus microservicios y un Gateway ligero, adecuado para empezar a aprender el patrón (De la Torre, 2018; Microsoft, s. f.-b).	Sistemas donde se quiere aplicar políticas mediante plugins delante de cualquier API REST, independientemente del lenguaje de los servicios (Kong Inc., s. f.-b).

Nota. Elaboración propia a partir de las fuentes citadas.

7. Conclusión: cuándo usar un Gateway y cuándo evitarlo

Conviene usarlo cuando hay varios microservicios y varios tipos de cliente, cuando los clientes hacen demasiadas llamadas para armar una sola vista, cuando se quiere centralizar seguridad, límites de uso y registro de solicitudes, o cuando las ubicaciones de los servicios cambian y no deben exponerse (Richardson, s. f.; Gravitee, s. f.).

Conviene evitarlo o posponerlo cuando la aplicación es pequeña o monolítica y un reverse proxy basta para enrutar, cuando el equipo no puede asumir la complejidad operativa de una pieza más (Richardson, s. f.), o cuando el diseño derivaría en un único Gateway central que concentre toda la lógica y se vuelva cuello de botella (De la Torre, 2018). Si se usa, conviene mantenerlo delgado: enrutamiento, políticas transversales y agregación sencilla, dejando la lógica de dominio en los servicios.

Conclusiones y recomendaciones

Conclusiones

- Un API Gateway ofrece una entrada única a los microservicios y oculta a los clientes la estructura interna del sistema.
- No es un simple reverse proxy: agrega composición de respuestas, adaptación por cliente y políticas transversales.
- Sus beneficios vienen con costos reales: complejidad, un salto de red adicional y el riesgo de convertirse en punto único de falla.
- Ocelot y Kong cubren las funciones principales de un Gateway, pero con enfoques distintos: Ocelot se integra a un proyecto ASP.NET Core y Kong es un Gateway independiente que se extiende con plugins.

Recomendaciones

- Elegir la herramienta según el stack del equipo y las funciones que realmente se necesitan.
- Evitar un Gateway único y sobrecargado: separar por cliente (BFF) o por equipo cuando el sistema crezca.
- Verificar en la documentación oficial cualquier función que las fuentes de este documento no confirmen para la herramienta elegida.
- Mantener registros (logs) del Gateway y de cada servicio, ya que una sola operación del cliente puede involucrar varias llamadas internas.

Bibliografía

Amazon Web Services. (s. f.). *Strangler fig pattern*. AWS Prescriptive Guidance. https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-decomposing-monoliths/strangler-fig.md

Apache APISIX. (s. f.). *API gateway for microservices: Architecture, patterns & best practices*. https://apisix.apache.org/learning-center/api-gateway-for-microservices/

De la Torre, C. (2018, 15 de mayo). *Designing and implementing API Gateways with Ocelot in .NET Core containers and microservices architectures*. Cesar de la Torre (Microsoft Developer Blogs). https://devblogs.microsoft.com/cesardelatorre/designing-and-implementing-api-gateways-with-ocelot-in-a-microservices-and-container-based-architecture/

Fowler, M. (2024, 22 de agosto). *Strangler fig*. martinfowler.com. https://www.martinfowler.com/bliki/StranglerFigApplication.html

Gravitee. (s. f.). *API gateway patterns for microservices*. https://gravitee.io/gateway-patterns.html

Kong Inc. (s. f.-a). *Health checks and circuit breakers reference* [Documentación de Kong Gateway 3.8.x]. Kong Docs. https://docs.konghq.com/gateway/3.8.x/how-kong-works/health-checks

Kong Inc. (s. f.-b). *Kong Gateway* [Documentación de la versión 2.8.x]. Kong Docs. https://docs.konghq.com/gateway/2.8.x/

Kong Inc. (s. f.-c). *Kong Gateway (OSS)* [Documentación de la versión 2.3.x]. Kong Docs. https://docs.jp.konghq.com/gateway-oss/2.3.x

Microsoft. (s. f.-a). *Gateway Aggregation pattern*. Azure Architecture Center. https://learn.microsoft.com/en-ie/Azure/architecture/patterns/gateway-aggregation

Microsoft. (s. f.-b). *Implement API Gateways with Ocelot*. En *.NET Microservices: Architecture for Containerized .NET Applications*. Microsoft Learn.

https://learn.microsoft.com/dotnet/architecture/microservices/multi-container-microservice-net-applications/implement-api-gateways-with-ocelot

Microsoft Azure. (s. f.). *Design patterns for microservices*. Azure Blog. https://azure.microsoft.com/en-us/blog/design-patterns-for-microservices/

Richardson, C. (s. f.). *Pattern: API Gateway / Backends for Frontends*. Microservices.io. https://microservices.io/patterns/apigateway.html

ThreeMammals. (s. f.). *Ocelot* [Repositorio de código]. GitHub. https://github.com/ThreeMammals/Ocelot